Hardware-platform aware pruning

Example of “Scalpel” technique

Sparsh Mittal

Acronyms

- MM = matrix-matrix multiplication
- SpMM = sparse matrix-matrix multiplication
- MVM = matrix-vector multiplication
- ILP = instruction-level parallelism
- MLP = memory-level parallelism
- TLP = thread-level parallelism
- Fmap = feature-map

Model-size aware and platform-aware pruning

- Conventional pruning techniques, such as deep-compression, seek to reduce “model-size”.
- However, the model-size reduction brought by them may not translate into corresponding reduction in execution latency.
- To mitigate this issue, we need to adapt the pruning technique according to the hardware platform.

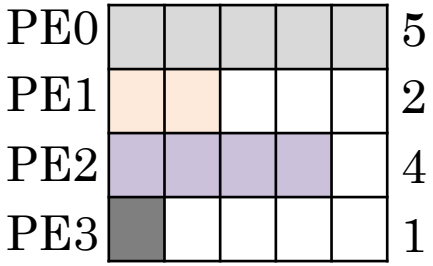
Naïve-pruning may lead to load-imbalance

- Consider a processor with four processing elements (PEs).

Weight matrix

PE0	W_{00}	W_{01}	0	W_{03}
PE1	0	0	W_{12}	0
PE2	0	W_{21}	0	W_{23}
PE3	0	0	0	0
PE0	0	W_{41}	0	W_{43}
PE1	0	0	W_{53}	0
PE2	W_{60}	0	0	W_{63}
PE3	0	W_{71}	0	0

Timeline



Makespan: **5**

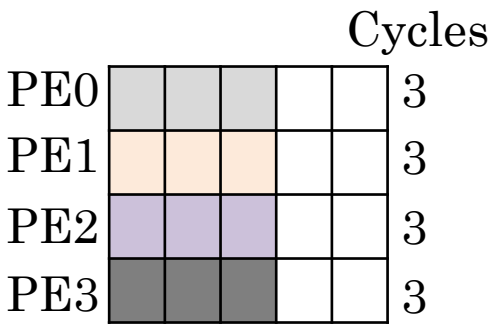
Naïve pruning leads to high makespan

Load-balance aware pruning

Weight matrix

PE0	W_{00}	0	0	W_{03}
PE1	0	0	W_{12}	0
PE2	0	W_{21}	0	W_{23}
PE3	0	0	W_{32}	0
PE0	0	0	0	W_{43}
PE1	0	0	W_{53}	W_{54}
PE2	W_{60}	0	0	
PE3	W_{70}	W_{71}	0	0

Timeline



Makespan: **3**

Load-balance aware pruning leads to short makespan
Lesson: pruning technique need to be aware of characteristics of processor system

Scalpel technique

Yu et al., “Scalpel: Customizing DNN pruning to the underlying hardware parallelism”, ISCA, 2017

Classification of processors based on parallelism

- **Low-parallelism** (e.g., microcontroller): they have in-order cores with no cache and limited storage. ARM Cortex-M4 has 3-stage pipeline and a 2-way SIMD design, 128KB SRAM and 512KB flash
- **Modest-parallelism** (e.g., out-of-order CPUs): they utilize ILP and MLP in addition to SIMD units. They also have deep cache hierarchy and very large memory capacity.
- **High-parallelism** (e.g., GPUs): they utilize TLP for further improving performance. Their throughput is sensitive to bandwidth and not to the DNN model size.

Platforms and Benchmarks

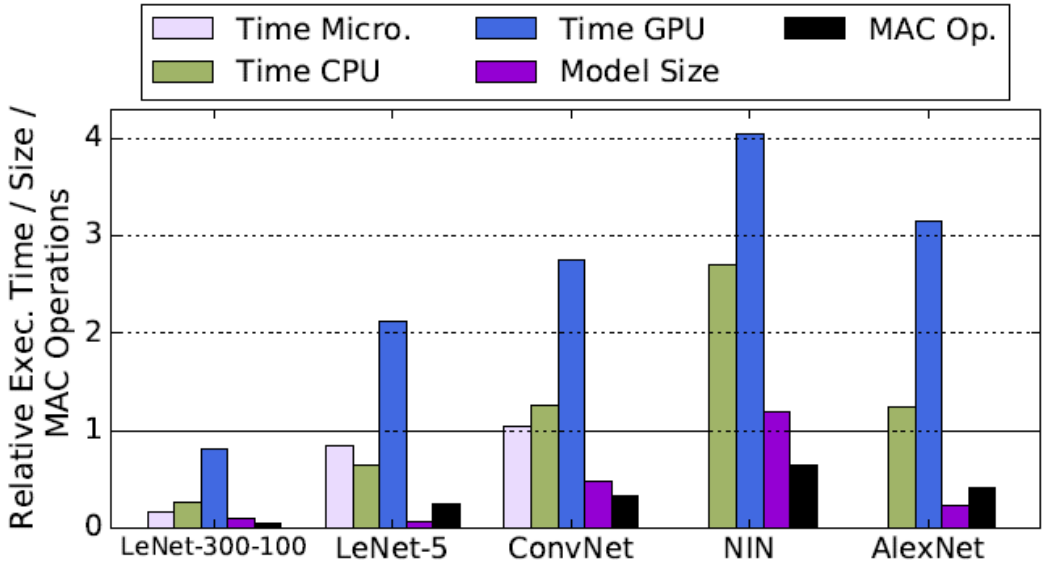
	Parallelism		
	Low	Moderate	High
Example	Micro-controller	CPU	GPU
Memory Hierarchy	No cache	Deep cache hierarchy	High bandwidth / long latency
Memory Size	~100KB SRAM	~8MB SRAM	2-12GB DRAM

Networks	Num of Layers		Test Dataset	Error Rate
	CONV	FC		
LeNet-300-100	0	3	MNIST	1.50%
LeNet-5	2	2		0.68%
ConvNet	3	1	CIFAR-10	18.14%
NIN	9	0		10.43%
AlexNet	5	3	ImageNet	19.73% (top-5)

- NIN and AlexNet are not tested on the microcontroller due to its limited storage size.

Motivational results with Deep Compression

- Relative execution time (Time Microcontroller/ CPU/GPU), model sizes, and MAC operations of the networks pruned by Deep Compression.



- Despite removing 80% of weights, pruning harms performance of 8 configs. On remaining configs, performance improvement was much lower than decrease in MACs.

Understanding performance impact of pruning

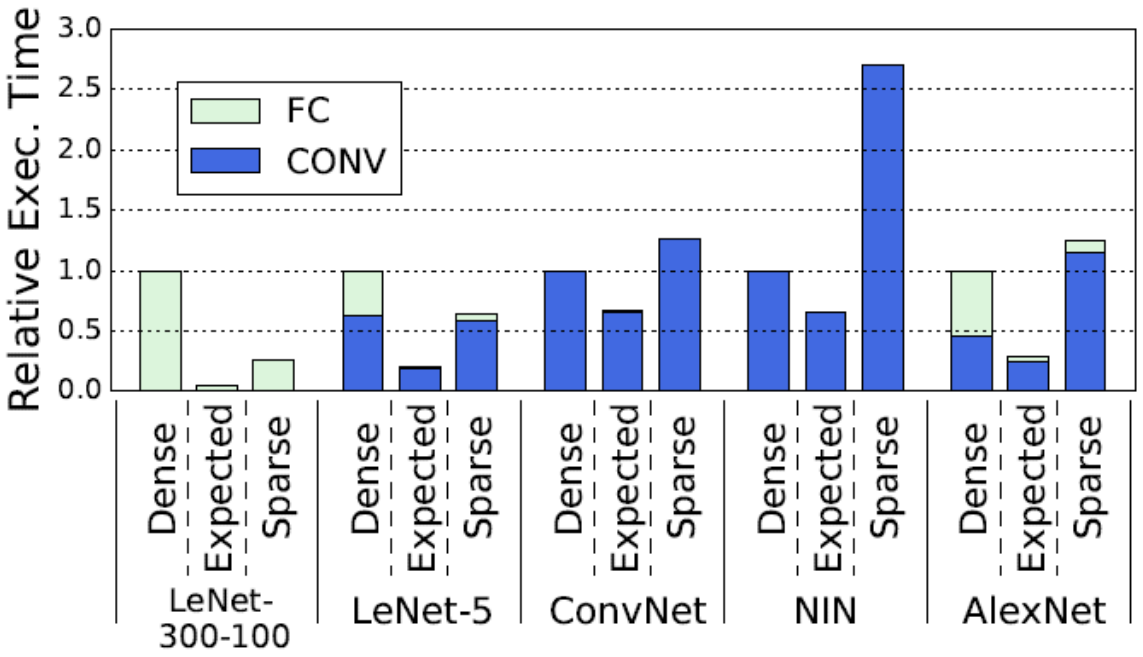
- Unlike dense matrices, sparse matrices are not regular.
- They require additional memory for storing the sparse format and computations for decoding it.
- **Microcontroller:** has a simple architecture => it does not hide the memory latency.
- => a decrease in model size due to pruning compensates the inefficiencies of SpMM.
- =>pruning improves performance on microcontroller
- **On GPUs**, pruning reduces the performance of all the networks, since with SpMM, optimizations such as memory-coalescing and matrix tiling cannot be applied

Understanding performance impact of pruning

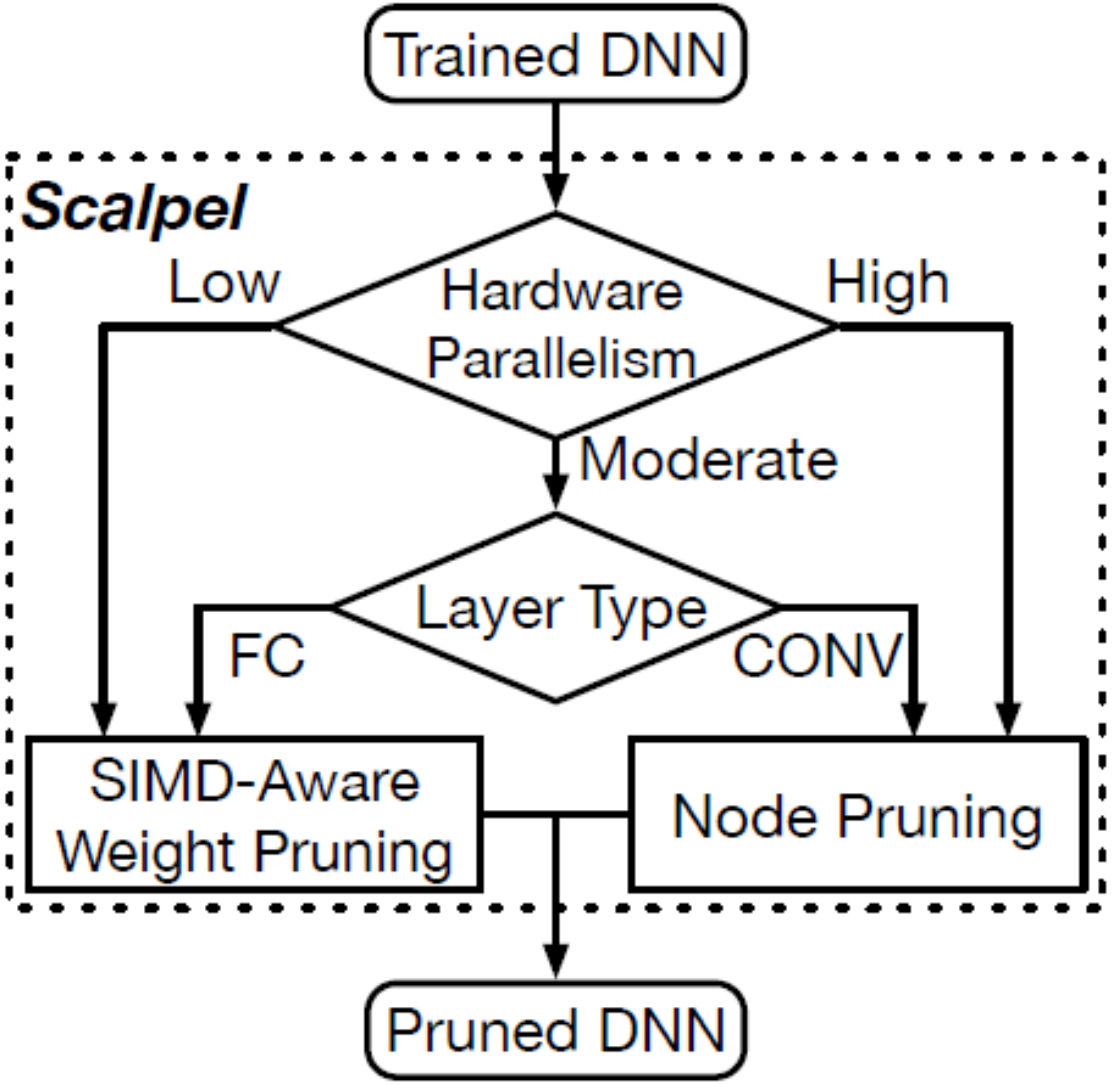
- **On CPUs,** impact of pruning depends on the computation breakdown in FC and CONV layers.
- Pruning improves performance of FC layers since they have high memory footprint and a decrease in memory accesses boosts performance of MVM operations.
- However, since CONV layers perform MM, where the data is reused several times, a decrease in memory footprint brings marginal benefit.
- Hence, pruning hurts performance of CONV layers on CPU.

Results on CPU

- Execution time breakdown of the original networks (Dense) and pruned networks (Sparse).
- Expected: shows expected execution time of pruned networks as the relative MAC operations remaining



Overall working of Scalpel



Scalpel technique on Microcontroller

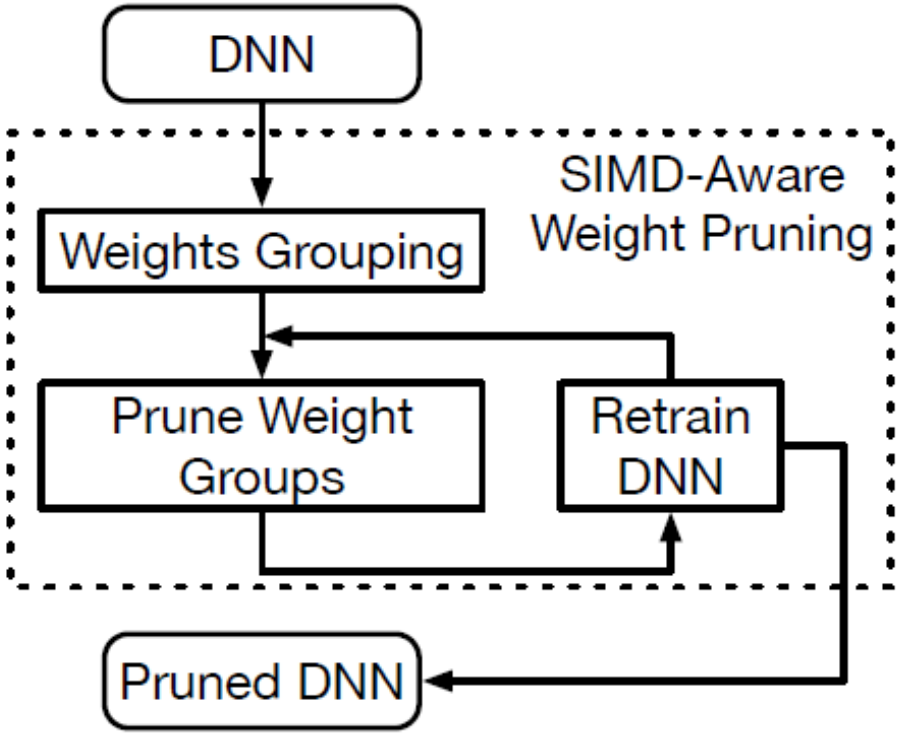
Background: computing root-mean square (RMS)

- Find RMS of [1, 5]
- Answer: sqrt(26/2) = 3.60

$$x_{\mathrm{RMS}} = \sqrt{\frac{1}{n}\left(x_1^2 + x_2^2 + \cdots + x_n^2\right)}.$$

- Find RMS of [7, 8, 2]
- Answer: sqrt($(7^2+8^2+2^2)/3$) = 6.24

SIMD-aware weight pruning (1 of 2)



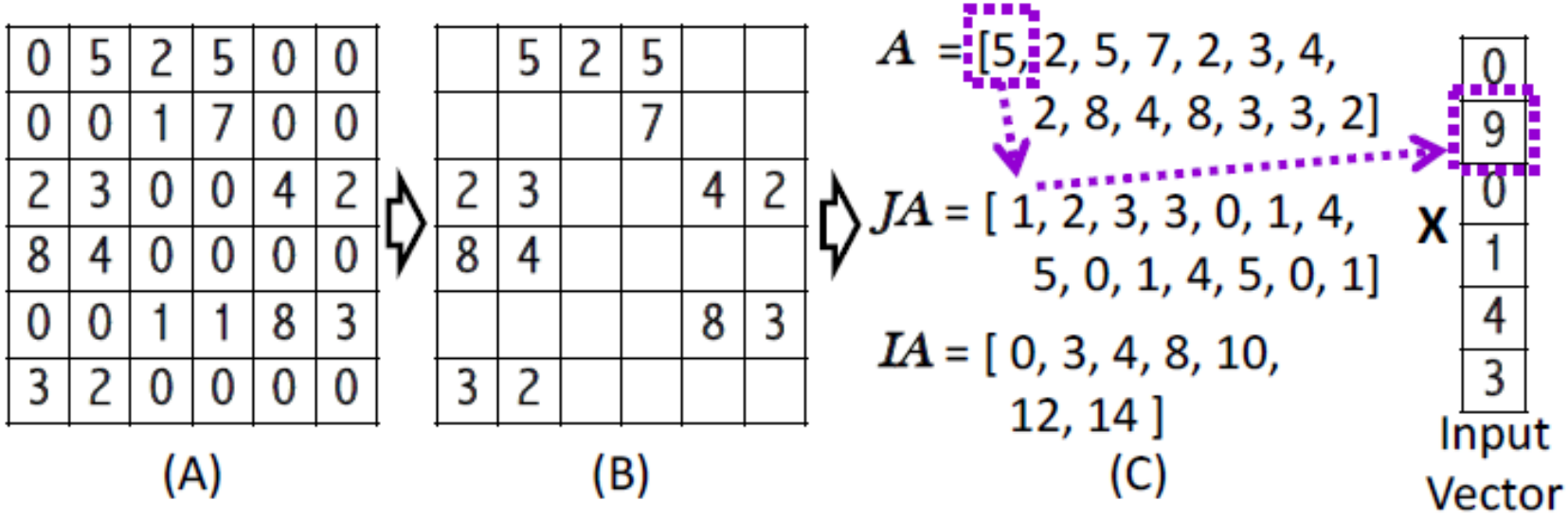
- First, weights are classified into aligned groups of size equal to SIMD width, e.g., 2 on a processor with 2-way SIMD.
- Root-mean-square of weights is found

- Groups with RMS below a threshold are pruned.
- Then, the pruned matrix is retrained.

SIMD-aware weight pruning (2 of 2)

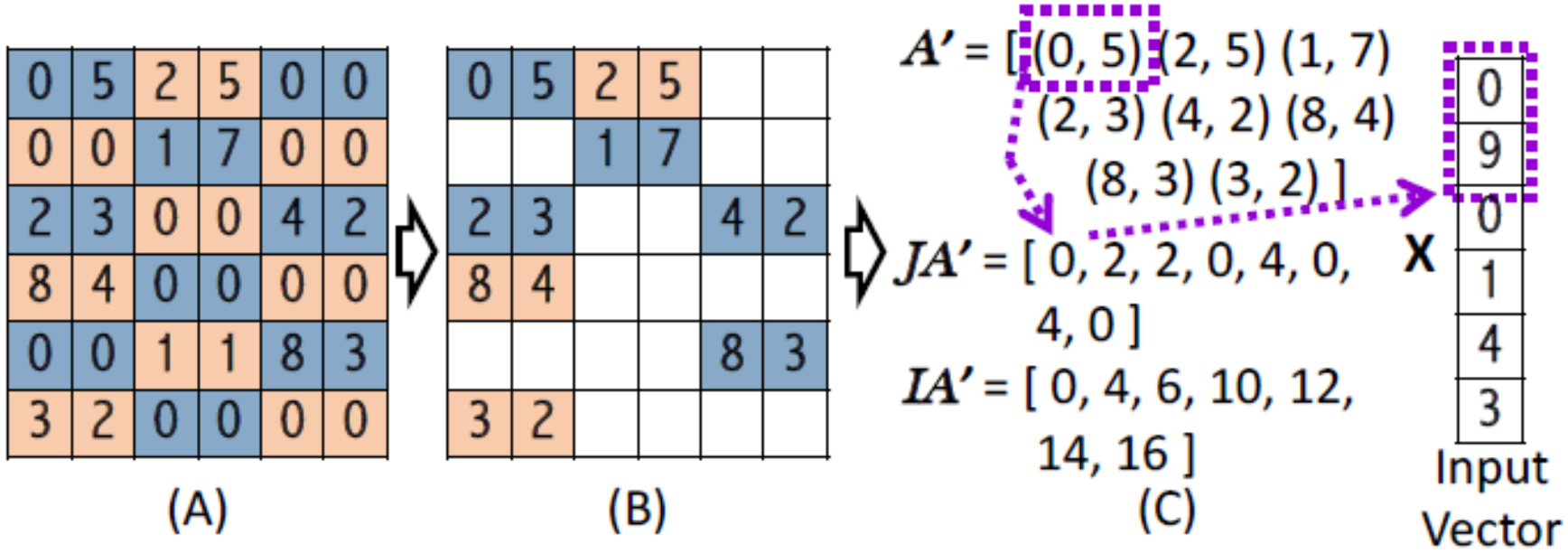
- Pruning and retraining are repeated till the accuracy of retrained DNN falls below that of original network.
- Layers with low redundancy or those not showing performance benefit from pruning are not pruned.
- Modified CSR format is used where column index of only the first element in every group is stored. This reduces the model size.

Traditional Pruning in Deep Compression



- (A) Dense weight matrix
- (B) Sparse weight matrix;
- (C) Compressed Sparse Rows (CSR) format for sparse matrices

SIMD-aware weight pruning

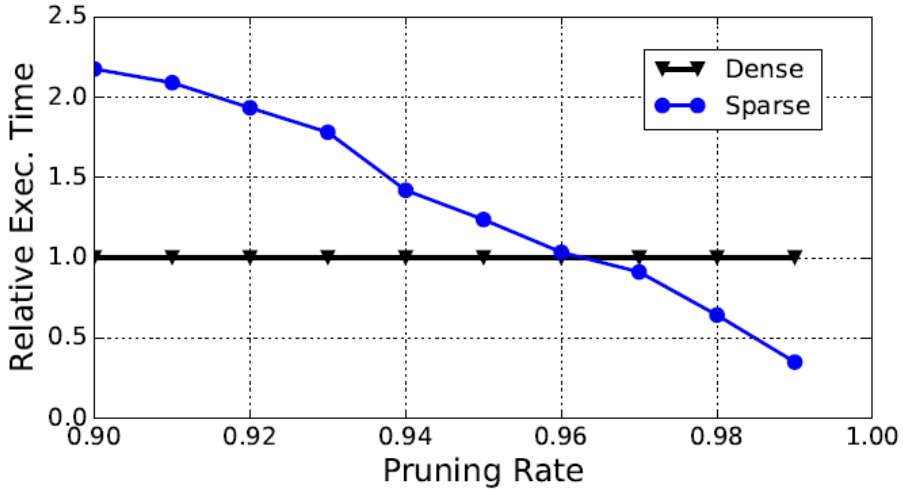


Since the inputs are present in contiguous locations, a single SIMD instruction is sufficient to load them. This reduces the number of instructions.

Scalpel technique on GPU

Weight pruning is not effective on GPU

- Relative execution time for SpMM (Sparse) on GPU wrt original execution time (Dense) on FC layer



- >96% weights need to be removed to achieve speedup.
- But removing that many weights leads to large loss in accuracy.

Scalpel uses node pruning on GPU

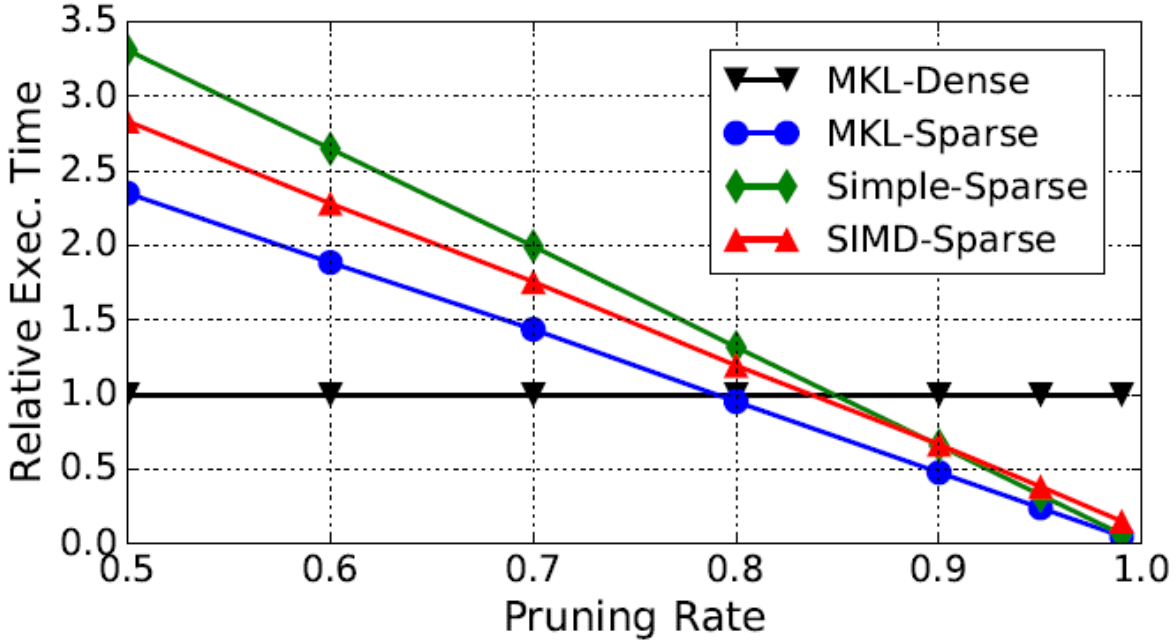
- On GPU, Scalpel performs node pruning.
- In FC layers, one neuron is taken as one node and in CONV layers, one fmap is taken as a node.
- Node pruning uses mask layers for selecting nodes which are unimportant and blocks their output.
- After mask layers are trained, the blocked nodes are removed, and after all redundant nodes are removed, mask layers are also removed.
- Finally, retraining is performed for obtaining the pruned network.

Scalpel uses node pruning on GPU

- Node pruning does not make the network sparse, and thus, the overheads of sparse network are not incurred.
- Node pruning reduces the layer-size but not as much as the weight pruning.
- Still, on GPUs, node pruning leads to larger throughput than the weight pruning technique.

Scalpel technique on CPU

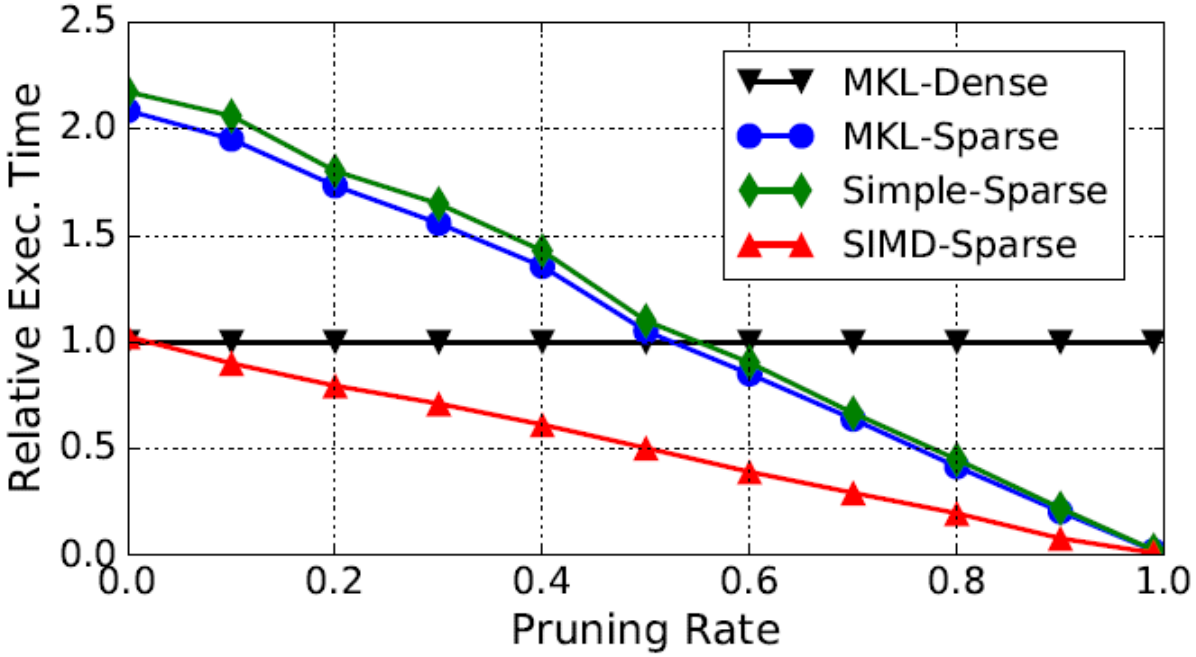
Results on CONV layers with SIMD-aware weight pruning



- To improve performance, >79% weights need to be pruned.
- Weight pruning not effective on CONV layers since all weights need to be reused in CONV operations.
- Use node pruning on CONV layers since it keeps regular structure of weight matrices

Results on FC Layer

- Relative execution time for sparse MVM (FC layers) on CPU. MKL-Dense/Sparse show results of dense and sparse weight matrix with the Intel MKL library.



Results on FC Layer

- SIMD-aware weight pruning removes weights in groups of 8 since CPU has 8-way SIMD.
- Compared to sparse MVM with MKL (MKL-Sparse), SIMD-aware weight pruning (SIMD-Sparse) improves performance greatly.
- Percentage of weights need to be removed for speedup decreases from 52% to 3%.
- Weight grouping increases spatial locality of reading weights and corresponding inputs.
- => For FC layers, SIMD-aware weight pruning is performed.

Results

They use batch size of 1 on microcontroller and CPU and of 50 on GPU.

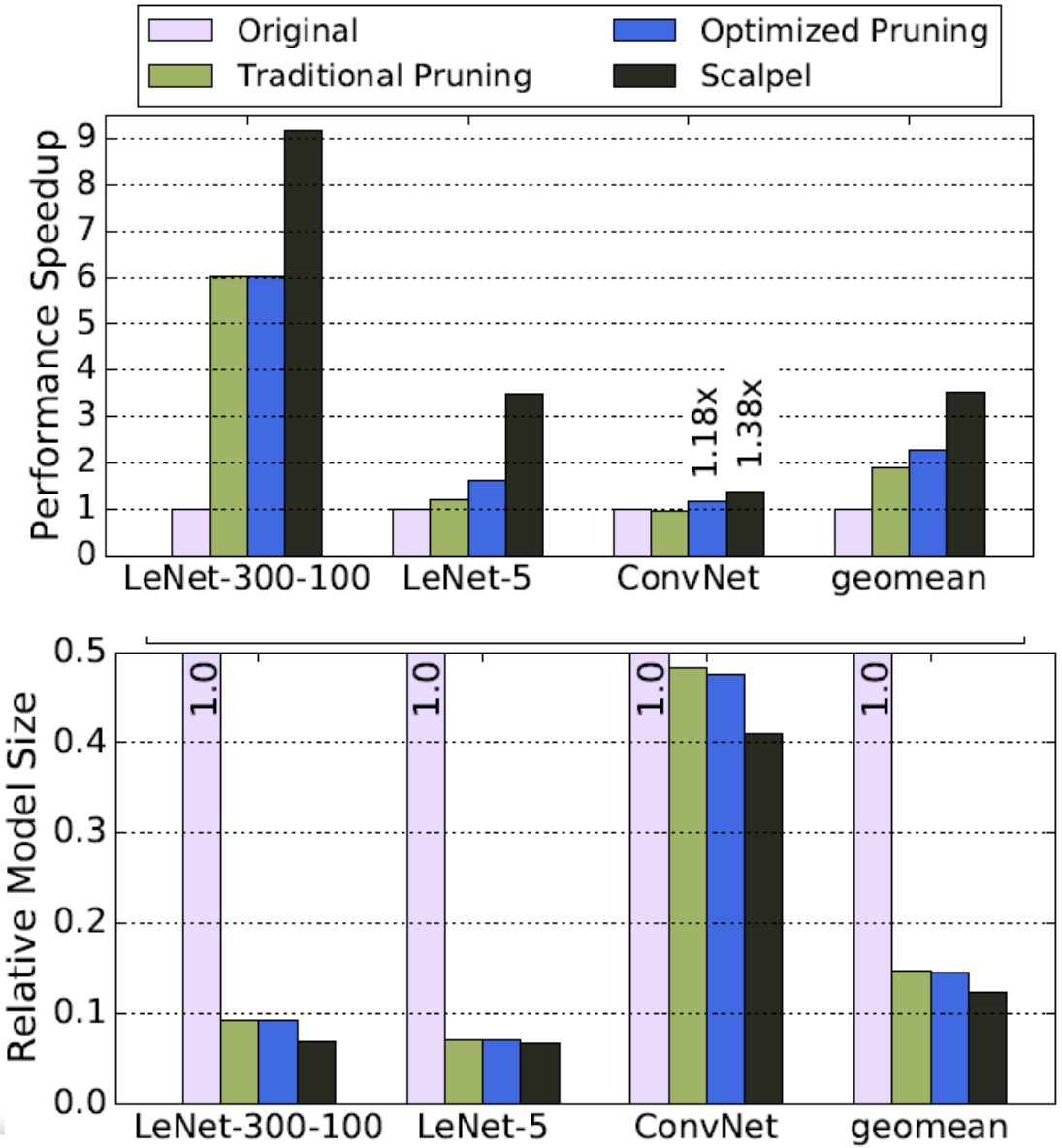
Experimentation Platform

- **CPU:** Intel Core i7-6700 (Skylake). Supports 8-way SIMD instructions for 32b floating-point numbers.
- MKL BLAS GEMV/ GEMM to implement the original dense model and CONV layers with node pruning. MKL Sparse BLAS CSRMV/ CSRMM for sparse models generated by existing weight pruning techniques
- **GPU:** NVIDIA GTX Titan X
- cuDNN to implement dense model and CONV layers with node pruning. cuBLAS GEMV/ GEMM for profiling performance of dense MM/MVM.
- cuSPARSE CSRMV/ CSRMM for sparse models

Summary of results with Scalpel

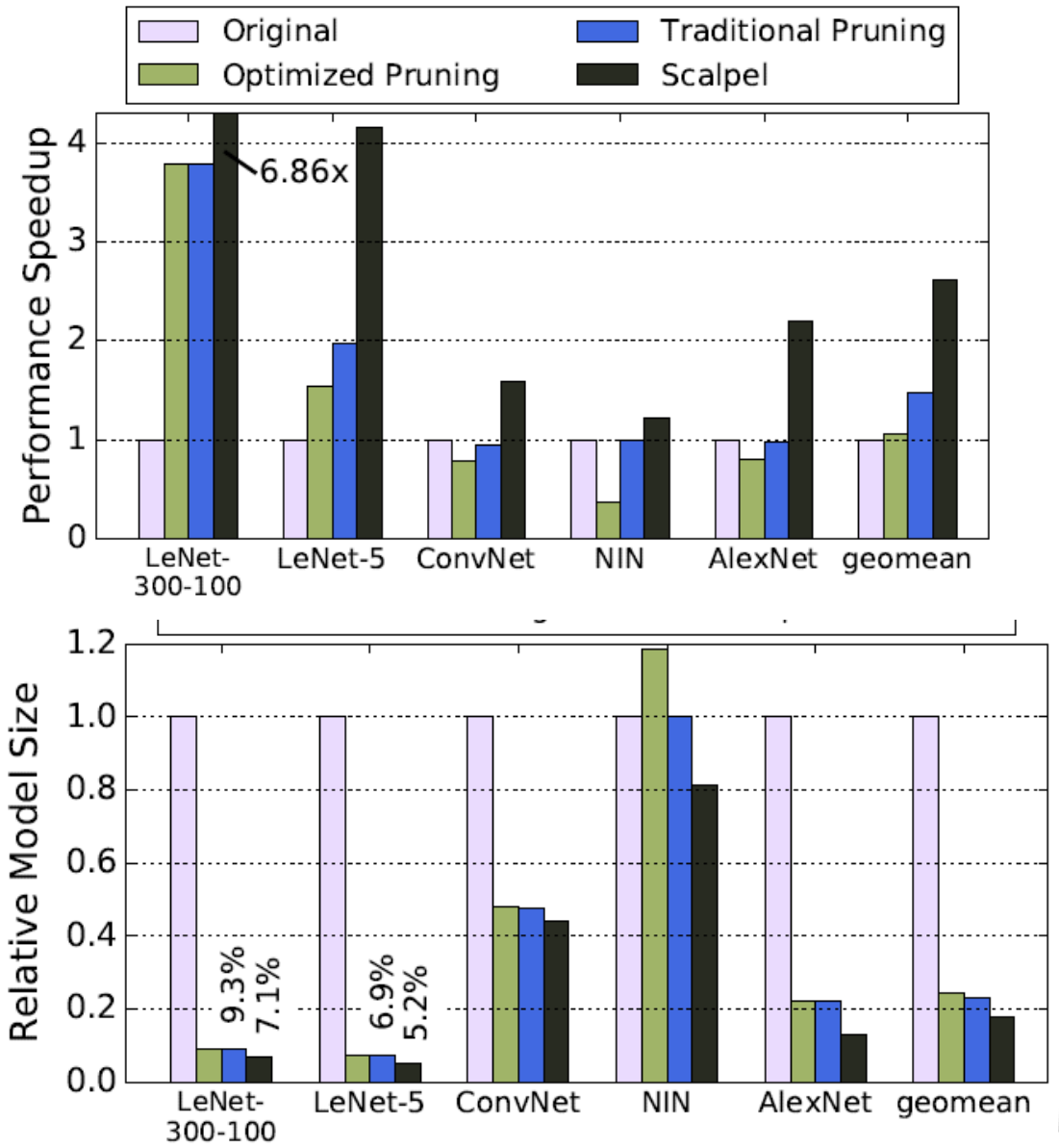
Hardware	DNNs	Speedup	Relative Size
Micor-controller	LeNet-300-100	9.17x	6.93%
	LeNet-5	3.51x	6.72%
	ConvNet	1.38x	40.95%
CPU	LeNet-300-100	6.86x	7.08%
	LeNet-5	4.15x	5.20%
	ConvNet	1.58x	44.28%
	NIN	1.22x	81.16%
	AlexNet	2.20x	13.06%
GPU	LeNet-300-100	1.08x	66.83%
	LeNet-5	1.59x	11.67%
	ConvNet	1.14x	45.40%
	NIN	1.17x	81.16%
	AlexNet	1.35x	76.52%

Speedup and model size on microcontroller

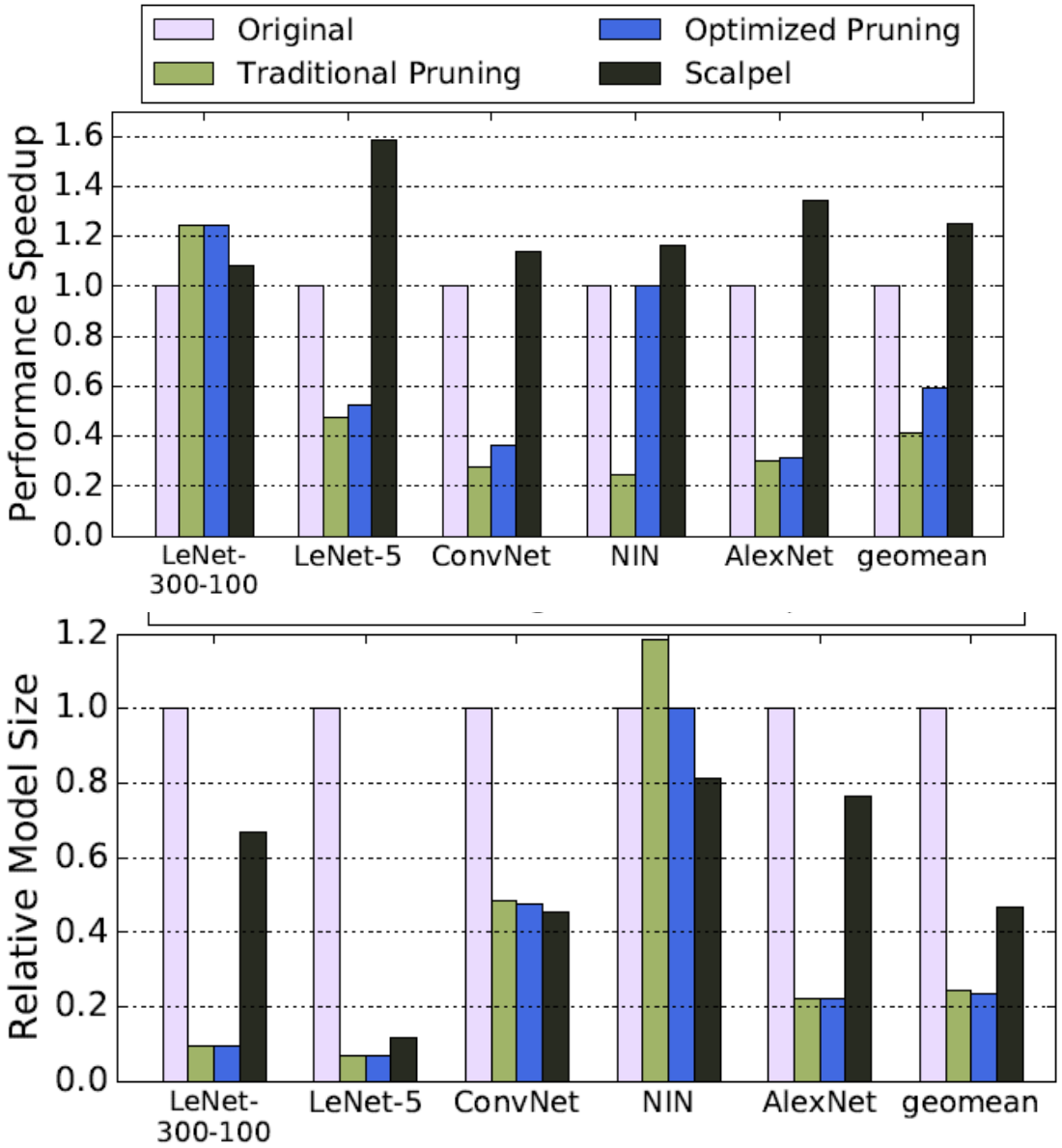


- Scalpel has better performance and less model size than other techniques

Speedup and model size on CPU



Speedup and model size on GPU



- For LeNet, speedup from Scalpel is low.
- LeNet has very tiny size and it is difficult to map computation efficiently onto GTX Titan X GPU.
- Also, a small change in matrix size leads to large difference in the matrix tiling strategy in cuBLAS.

Accuracy results

- Scalpel pruning does not lead to accuracy loss compared to the original DNNs